Zetascope, Inc.

GPU Engineer Technical Interview

Brute-Force Track-Before-Detect on CUDA

February 24, 2026

1 Introduction

1.1 Background

Track-Before-Detect (TBD) is a signal processing paradigm for detecting objects that are too faint to observe in any single measurement. Instead of detecting first and tracking second (the classical approach), TBD accumulates evidence across many measurements along candidate motion hypotheses, exploiting the fact that signal grows as $\mathcal{O}(N)$ while noise grows as $\mathcal{O}(\sqrt{N})$ for N integrated frames.

In the context of **space domain awareness**, TBD enables detection of extremely faint resident space objects (RSOs) at cislunar distances, where the object may contribute only a handful of photons per frame against a noisy background.

1.2 The Shift-and-Stack Algorithm

The simplest TBD method is **brute-force shift-and-stack**: for every candidate trajectory defined by a starting pixel position (x_0, y_0) and constant velocity (v_x, v_y), sum the pixel values along the predicted path across all frames. The trajectory producing the highest sum (the *test statistic*) is the detection.

Formally, given a sequence of N frames $\{I_t\}_{t=0}^{N-1}$, the test statistic is:

$$\boxed{T(x_0, y_0, v_x, v_y) \;=\; \sum_{t=0}^{N-1} I_t[\,\mathrm{round}(y_0 + v_y \cdot t),\ \mathrm{round}(x_0 + v_x \cdot t)\,]} \tag{1}$$

where $I_t[r, c]$ denotes the pixel value at row r, column c of frame t. The detection is:

$$(\hat{x}_0,\ \hat{y}_0,\ \hat{v}_x,\ \hat{v}_y) \;=\; \arg\max_{(x_0, y_0, v_x, v_y)} T(x_0, y_0, v_x, v_y) \tag{2}$$

Why Brute-Force?

While more sophisticated algorithms (temporal-velocity pyramids, FFT-based methods, multi-resolution search) exist to accelerate TBD, the brute-force approach is the *gold standard* baseline: it is exact, trivially parallelizable, and serves as the correctness reference for all optimized methods. Understanding how to efficiently map this embarrassingly parallel problem onto GPU hardware is a fundamental skill.

2 Problem Specification

2.1 Synthetic Data Model

Each frame is generated as independent Poisson noise with a faint moving point source injected along a ground-truth trajectory:

$$I_t[r, c] \;\sim\; \text{Poisson}(\lambda_{\text{bg}}) + \delta_{r,r_t}\,\delta_{c,c_t}\,\text{Poisson}(\lambda_{\text{obj}}) \tag{3}$$

where $(c_t, r_t) = \big(\text{round}(x_0^* + v_x^* t),\ \text{round}(y_0^* + v_y^* t)\big)$ is the true object position at frame t.

2.2 Problem Parameters

Table 1 lists all fixed parameters for this exercise.

Table 1: Problem parameters.

Parameter	**Symbol**	**Value**	**Unit**
Image width	N_x	256	pixels
Image height	N_y	256	pixels
Number of frames	N	16	frames
Background rate	λ_{bg}	100.0	photons/pixel/frame
Object signal	λ_{obj}	20.0	photons/frame
True starting position	$(x_0^*,\ y_0^*)$	(128, 100)	pixels
True velocity	$(v_x^*,\ v_y^*)$	(1.5, −0.75)	pixels/frame
Velocity search min	v_{min}	−3.0	pixels/frame
Velocity search max	v_{max}	+3.0	pixels/frame
Velocity step size	Δv	0.25	pixels/frame
RNG seed	—	42	—

2.3 Signal-to-Noise Analysis

The per-frame signal-to-noise ratio for a Poisson background is:

$$\text{SNR}_1 \;=\; \frac{\lambda_{\text{obj}}}{\sqrt{\lambda_{\text{bg}}}} \;=\; \frac{20}{\sqrt{100}} \;=\; 2.0 \tag{4}$$

This is well below typical single-frame detection thresholds (SNR ≥ 5). After coherent integration across $N = 16$ frames, the stacked SNR becomes:

$$\text{SNR}_N \;=\; \text{SNR}_1 \cdot \sqrt{N} \;=\; 2.0 \times \sqrt{16} \;=\; 8.0 \tag{5}$$

which is comfortably above threshold — but *only if the correct trajectory is used for integration.* This is the fundamental premise of TBD.

2.4 Search Space Complexity

The velocity grid contains:

$$N_v \;=\; \left\lfloor \frac{v_{\max} - v_{\min}}{\Delta v} \right\rfloor + 1 \;=\; \left\lfloor \frac{6.0}{0.25} \right\rfloor + 1 \;=\; 25 \tag{6}$$

velocity steps per axis, giving $N_v^2 = 625$ velocity hypotheses. The total number of candidate trajectories is:

$$N_{\text{traj}} \;=\; N_x \cdot N_y \cdot N_v^2 \;=\; 256 \times 256 \times 625 \;=\; 40{,}960{,}000 \;\approx\; 41\text{M} \tag{7}$$

Each trajectory evaluation requires $N = 16$ pixel lookups, yielding:

$$N_{\text{reads}} \;=\; N_{\text{traj}} \times N \;=\; 40{,}960{,}000 \times 16 \;=\; 655{,}360{,}000 \;\approx\; 655\text{M pixel reads} \tag{8}$$

3 Algorithm Pseudocode

Algorithm 1 Brute-Force Shift-and-Stack TBD

Require: Frames $\{I_t\}_{t=0}^{N-1}$, velocity grid $\mathcal{V} = \{v_{\min}, v_{\min} + \Delta v, \ldots, v_{\max}\}$
Ensure: Detection $(\hat{x}_0, \hat{y}_0, \hat{v}_x, \hat{v}_y)$ and test statistic $\hat{T}$

```
T̂ ← −∞
for y_0 = 0 to N_y − 1 do
    for x_0 = 0 to N_x − 1 do
        for v_x ∈ V do
            for v_y ∈ V do
                T ← 0;   valid ← TRUE
                for t = 0 to N − 1 do
                    p_x ← round(x_0 + v_x · t)
                    p_y ← round(y_0 + v_y · t)
                    if p_x ∉ [0, N_x) or p_y ∉ [0, N_y) then
                        valid ← FALSE;   break
                    end if
                    T ← T + I_t[p_y, p_x]
                end for
                if valid and T > T̂ then
                    T̂ ← T;   (x̂_0, ŷ_0, v̂_x, v̂_y) ← (x_0, y_0, v_x, v_y)
                end if
            end for
        end for
    end for
end for
return (x̂_0, ŷ_0, v̂_x, v̂_y, T̂)
```

Boundary Handling

If the predicted position (p_x, p_y) at *any* frame t falls outside the image boundaries $[0, N_x) \times [0, N_y)$, the entire trajectory is marked invalid and skipped. This is physically correct: partial trajectories would have reduced integrated signal and should not compete with full-length trajectories.

4 Binary Data Format

The Python script `tbd_problem.py` generates two binary files. Your CUDA implementation must read these files.

4.1 `tbd_frames.bin`

Table 2: Frame data binary layout.

Offset (bytes)	Type	Count	Description
0	`int32`	1	N_x (image width)
4	`int32`	1	N_y (image height)
8	`int32`	1	N (number of frames)
12	`float32`	$N \times N_y \times N_x$	Pixel data

The pixel data is stored in **frame-major, row-major** order:

$$\texttt{offset}(t, r, c) \;=\; 12 + 4 \times \big(t \cdot N_y \cdot N_x + r \cdot N_x + c\big) \text{ bytes} \tag{9}$$

4.2 `tbd_params.bin`

Table 3: Parameter binary layout.

Offset (bytes)	Type	Count	Description
0	`float32`	1	$v_{\min}$
4	`float32`	1	$v_{\max}$
8	`float32`	1	Δv

5 Your Task

5.1 Deliverables

Implement a **CUDA/C++ program** that:

1. Reads `tbd_frames.bin` and `tbd_params.bin`
2. Performs the brute-force shift-and-stack search (Eq. 1) entirely on the GPU
3. Reports the detected trajectory $(\hat{x}_0, \hat{y}_0, \hat{v}_x, \hat{v}_y)$ and test statistic $\hat{T}$

4. Reports wall-clock kernel execution time using CUDA events
5. Compiles with `nvcc` (any SM architecture ≥ 7.0)

5.2 Setup Instructions

1. Generate the test data:

```
python3 tbd_problem.py
```

2. Verify that `tbd_frames.bin`, `tbd_params.bin`, and `tbd_reference.txt` are created.
3. Implement your solution and compile:

```
nvcc -O3 -arch=sm_86 -o tbd_solution your_solution.cu
./tbd_solution tbd_frames.bin tbd_params.bin
```

4. Your output must match `tbd_reference.txt`.

5.3 Reference Answer

Table 4: Expected detection result.

Quantity	**Value**
$\hat{x}_0$	128
$\hat{y}_0$	100
$\hat{v}_x$	1.50 pixels/frame
$\hat{v}_y$	−0.75 pixels/frame
$\hat{T}$	1881.0

6 GPU Parallelization Considerations

This section describes the key considerations for an efficient CUDA implementation. Candidates are expected to address *at least* items 6.1–6.4; items 6.5–6.6 are differentiators for strong candidates.

6.1 Thread-to-Work Mapping

The search has a natural 4D structure: (x_0, y_0, v_x, v_y). A common and effective strategy is to assign one GPU thread per starting pixel (x_0, y_0), with each thread looping over all velocity hypotheses. This gives $256 \times 256 = 65{,}536$ threads — sufficient for good occupancy on modern GPUs.

Thread Block Configuration

A 16×16 thread block (256 threads) is a natural choice matching the 2D spatial structure. With 16×16 blocks in the grid, this maps directly to the 256×256 pixel space.

6.2 Memory Hierarchy

- **Constant memory** (`__constant__`): Ideal for the velocity grid. All threads read the *same* velocity values simultaneously (broadcast pattern), making constant memory's single-broadcast cache architecture a perfect fit. The velocity grid (25×4 bytes $= 100$ bytes) fits trivially in the 64 KB constant memory limit.
- **Read-only / texture cache** (`__ldg()`): Frame data is read-only and accessed with spatial locality (neighboring threads read nearby pixels). Using `__ldg()` intrinsics routes reads through the read-only data cache, which is optimized for this pattern.
- **Registers**: The running sum T, validity flag, and best-so-far tracking variables should reside in registers. With $N_v^2 = 625$ iterations per thread, register pressure is low.

6.3 Global Memory Access Patterns

For adjacent threads (neighboring x_0 values), the predicted pixel positions at frame t are:

$$p_x^{(i)} = \text{round}(x_0^{(i)} + v_x \cdot t) \tag{10}$$

Since x_0 values differ by 1 pixel, the read addresses differ by 1 float — this is **coalesced** for small velocities and nearly coalesced for larger ones, which is favorable for global memory throughput.

6.4 Reduction

After the search kernel, each pixel has a local best statistic. A parallel reduction is needed to find the global maximum across all 256×256 pixels. This can be implemented as:

1. A shared-memory tree reduction within each thread block
2. A final reduction over block-level results (on host or with a second kernel)

6.5 Shared Memory Tiling (Advanced)

For each velocity hypothesis and frame t, threads in a block access a *shifted* tile of the frame. One could preload the relevant tile (plus halo) into shared memory before the inner loop. However, given that the `__ldg()` cache already handles this pattern well and the frame data (4 MB) fits comfortably in L2 cache on modern GPUs, the benefit may be marginal. A candidate who *analyzes* this tradeoff (with or without implementing it) demonstrates strong understanding.

6.6 Further Optimizations (Advanced)

- **Early termination**: Skip velocity hypotheses where the first few frames already guarantee the trajectory exits the image.
- **Velocity-parallel decomposition**: Instead of one thread per pixel looping over velocities, launch threads over (x_0, y_0, v_x) and loop over v_y (or vice versa), increasing parallelism at the cost of a more complex reduction.
- **Half-precision accumulation**: Use `__half2` arithmetic for frame reads, accumulating in FP32. This doubles memory bandwidth utilization on architectures supporting native FP16.

- **Warp-level primitives**: Use `__shfl_down_sync()` for the first stages of reduction, avoiding shared memory bank conflicts.
- **Streams and overlap**: Overlap data transfer with computation using CUDA streams (less relevant here since the data is small, but shows awareness of the technique).

7 Performance Targets

Table 5: Performance reference points.

Implementation	Est. Time	Speedup	Notes
Python (pure loops)	∼3 hours	1×	Algorithm 1 verbatim
NumPy (vectorized)	∼3 seconds	∼3600×	Vectorize over (x_0, y_0)
CUDA (basic)	$<$ 100 ms	>100,000×	Correct thread mapping
CUDA (optimized)	$<$ 20 ms	>500,000×	Memory hierarchy tuned

An implementation that produces the correct detection in 200 ms is **far more valuable** than one that runs in 5 ms but gives the wrong answer. Verify correctness against `tbd_reference.txt` before optimizing.

8 Evaluation Rubric

Table 6: Scoring rubric (100 points total).

Category	Points	Criteria
Correctness	30	Detected trajectory matches reference. Proper boundary handling. No race conditions or undefined behavior.
Performance	25	Kernel time under 100 ms (baseline pass). Bonus for sub-20 ms. Proper use of CUDA events for timing.
Parallelization Strategy	20	Sensible thread-to-work mapping with good occupancy. Block size justified. Scalability to larger problems.
Memory Optimization	15	Use of constant memory for velocity grid. Awareness of coalescing. Use of `__ldg()` or texture cache.
Code Quality	10	Error handling (`cudaGetLastError`). Readable code structure. Clean build with no warnings. Comments on design choices.

9 Context: Why This Matters

This interview problem is a simplified version of the real computational challenge at Zetascope. In the operational system:

- Images are 4096×4096 (or larger) from qCMOS sensors
- Velocity grids may contain $\sim$10,000+ hypotheses
- Processing must happen in *real-time* within the sensor's integration window
- The system runs on space-qualified embedded GPUs (NVIDIA Jetson AGX Orin class) with strict power and thermal budgets
- Objects of interest may contribute fewer than **one photon per frame** on average, requiring integration across hundreds of frames

Efficiently mapping brute-force TBD to GPU hardware is the foundation upon which all of our more advanced algorithmic accelerations (temporal-velocity pyramids, multi-resolution search, sensor-algorithm co-optimization) are built. The skills tested here — thread mapping, memory hierarchy exploitation, parallel reduction, and performance measurement — are directly applicable to the production system.

Zetascope, Inc. — Confidential Interview Material
Do not distribute.